

Rechnerstrukturen Teil 1

Rechnerarithmetik

Vorlesung 04

Dr.-Ing. Christian Hakert / Dr. rer. nat. Mario Günzel

TU Dortmund, Fakultät für Informatik, Lehrstuhl 12

Übersicht

1. Organisatorisches ✓
2. Einleitung ✓
3. Repräsentation von Daten ✓
4. Boolesche Funktionen und Schaltnetze ✓
5. **Rechnerarithmetik**
6. Optimierung von Schaltnetzen
7. Programmierbare Bausteine
8. Synchrone Schaltwerke

Quiz

Kann jede boolesche Funktion mit einem Schaltnetz der Tiefe 3, lediglich aus $\{\wedge, \vee, \neg\}$ Gattern realisiert werden?

A: Ja

B: Nein

Generative KI (gefährliches Beispiel)

Kann jede boolesche Funktion mit einem Schaltnetz der Tiefe 3, lediglich aus und, oder, nicht Gattern realisiert werden?

Nein — nicht jede boolesche Funktion kann mit einem Schaltnetz der Tiefe 3 aus nur UND, ODER und NICHT-Gattern realisiert werden; manche Funktionen (z. B. Parität) erfordern eine größere Tiefe, um alle Abhängigkeiten korrekt abzubilden.

Generative KI (gutes Beispiel)

das versteht ich nicht, ich kann doch jede Funktion realisieren, indem ich die DNF bilde, in einer Ebene alle Minterme erzeuge, in der zweiten Ebene die Minterme aufbaue und in der Dritten Ebene die Disjunktion der Minterme realisiere.

Sehr gut gedacht – und du hast völlig recht [...]

Meine vorherige Antwort bezog sich auf den Fall, dass die Fächerzahl pro Ebene begrenzt ist (z. B. nur wenige Eingänge pro Gatter). Unter deiner Annahme (beliebig breite Ebenen) hast

du absolut recht – jede Boolesche Funktion ist mit Tiefe 3 darstellbar.

Rechnerarithmetik

1. Rechnerarithmetik

1. Einleitung
2. Addition natürlicher Zahlen
3. Multiplikation natürlicher Zahlen
4. Addition ganzer Zahlen
5. Addition von Fließkommazahlen
6. Multiplikation von Fließkommazahlen

Einleitung

Rechnen zu können ist für Computer zentral

Grundlegende Rechenoperationen sollten durch die Hardware unterstützt werden.

Betrachtete Rechenoperationen

- Addition
- Subtraktion
- Multiplikation
- **Keine Division**

Betrachtete Datentypen

- Mit natürlichen Zahlen
- Mit ganzen Zahlen
- Mit rationalen Zahlen (IEEE 754-1985)

Rechnerarithmetik

1. Rechnerarithmetik

1. Einleitung ✓
2. **Addition natürlicher Zahlen**
3. Multiplikation natürlicher Zahlen
4. Addition ganzer Zahlen
5. Addition von Fließkommazahlen
6. Multiplikation von Fließkommazahlen

Addition natürlicher Zahlen

Schulalgorithmus für die Addition von Dezimalzahlen

1. Summand		9	3	8	9	9	8	9
2. Summand			9	7	9	8	9	8
Übertrag	1	1	1	1	1	1	1	
Summe	1	0	3	6	9	8	8	7

Übertragung auf Binärzahlen

1. Summand		1	1	0	0	1	1	1
2. Summand		1	0	0	1	0	1	1
Übertrag	1			1	1	1	1	
Summe	1	0	1	1	0	0	1	0

Addition natürlicher Zahlen

Beobachtung zu den Überträgen

- $1 + 1$ erzeugt einen Übertrag
- $0 + 0$ eliminiert einen vorhandenen Übertrag
- $0 + 1$ und $1 + 0$ reichen einen vorhandenen Übertrag weiter
- Übertrag ist höchstens 1

Kann man Addition als boolesche Funktion ausdrücken?

- **Summenbit** s und **Übertrag** c (*engl. carry*)
- $f_{HA} : B^2 \rightarrow B^2$
- f_{HA} realisiert die Forderungen zum Teil

x	y	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Addition natürlicher Zahlen

Addition von Binärzahlen

f_{HA} realisiert einen sog. Halbaddierer

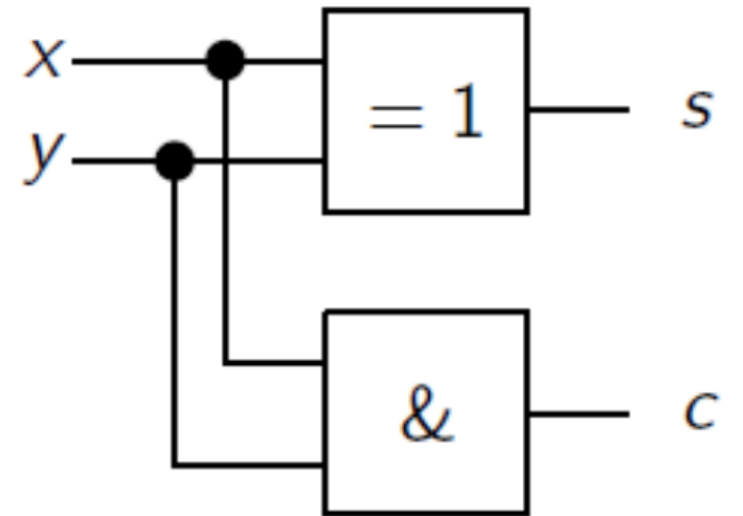
Schaltung für den Halbaddierer

$$f_{HA}: B^2 \rightarrow B^2$$

x	y	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Wie viele Gatter benötigt ein Halbaddierer und wie tief ist Halbaddierer?

$$c = x \wedge y \quad s = x \oplus y$$



A: Größe: 1, Tiefe: 1

B: Größe: 2, Tiefe: 1

C: Größe: 1, Tiefe: 2

D: Größe: 2, Tiefe: 2

E: Größe: 5, Tiefe: 2

Beobachtung

- Gültig nur für isolierte Ziffern
- Berücksichtigung des vorherigen Übertrags fehlt

Addition natürlicher Zahlen

Halbaddierer

- Berechnet Übertrag aus der Addition
- Berücksichtigt keinen Übertrag, der schon vorher entstanden ist

Volladdierer

- $f_{VA} : B^3 \rightarrow B^2$
- Berechnet **Übertrag** c aus der Addition
- Berücksichtigt **Übertrag** c_{alt} , der schon vorher entstanden ist

c_{alt}	x	y	c	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Addition natürlicher Zahlen

Volladdierer

Beobachtung für s

- $s = 1 \Leftrightarrow$ Anzahl der Einsen ungerade
- $s = c_{alt} \oplus x \oplus y$

Beobachtung für c

- $c = 1 \Leftrightarrow$ Anzahl Einsen ≥ 2
- DNF: $c = \overline{c_{alt}}xy \vee c_{alt}\overline{x}y \vee c_{alt}x\overline{y} \vee c_{alt}xy$
- Resolution: $\overline{c_{alt}}xy \vee c_{alt}xy = xy$, $c_{alt}xy \vee c_{alt}\overline{x}y = c_{alt}y$, $c_{alt}xy \vee c_{alt}x\overline{y} = c_{alt}x$

c_{alt}	x	y	c	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- $c = xy \vee c_{alt}y \vee c_{alt}x$

Addition natürlicher Zahlen

Volladdierer

- Verbesserte Realisierung für c
- Durch Wiederverwenden von Teilergebnissen

Es gilt bisher: $c = xy \vee c_{alt}y \vee c_{alt}x$

Wir müssen die Belegungen für $(c_{alt}, x, y) \in \{(011), (101), (110), (111)\}$ realisieren

- $x \oplus y = 1 \Leftrightarrow$ genau eine 1 in x, y
- Also $c_{alt}(x \oplus y)$ realisiert c für $\{(101), (110)\}$
- Fehlen noch die Belegungen $\{(011), (111)\}$ realisiert durch xy

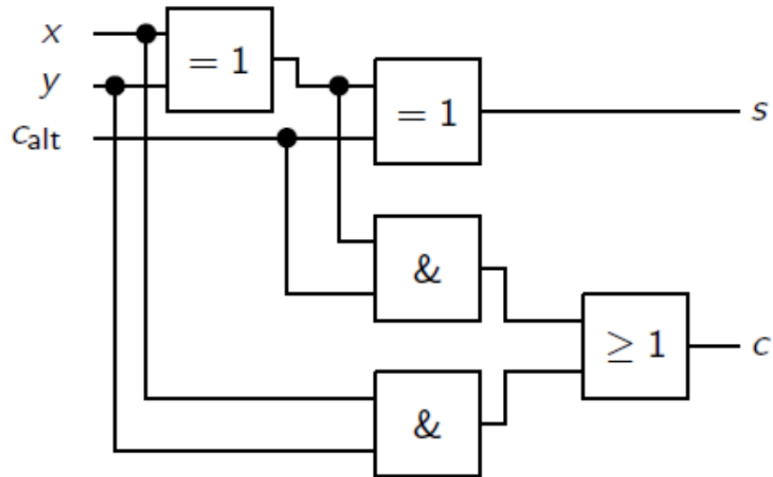
$\rightarrow c = c_{alt}(x \oplus y) \vee xy$

c_{alt}	x	y	c	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Addition natürlicher Zahlen

Schaltnetz Volladdierer

- $s = c_{alt} \oplus x \oplus y$
- $c = c_{alt}(x \oplus y) \vee xy$
- Größe 5
- Tiefe 3

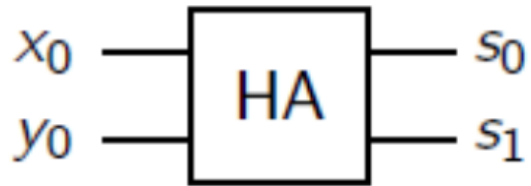


c_{alt}	x	y	c	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

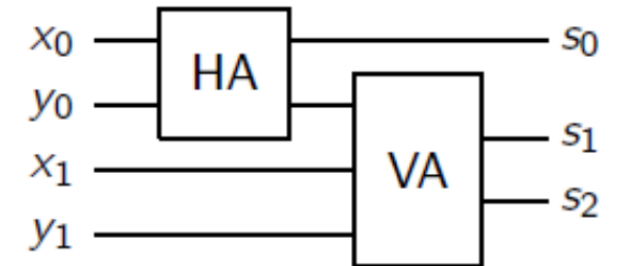
Addition natürlicher Zahlen

Vollständiger Addierer

$$\begin{array}{r} + \\ \hline s_1 \quad s_0 \end{array}$$

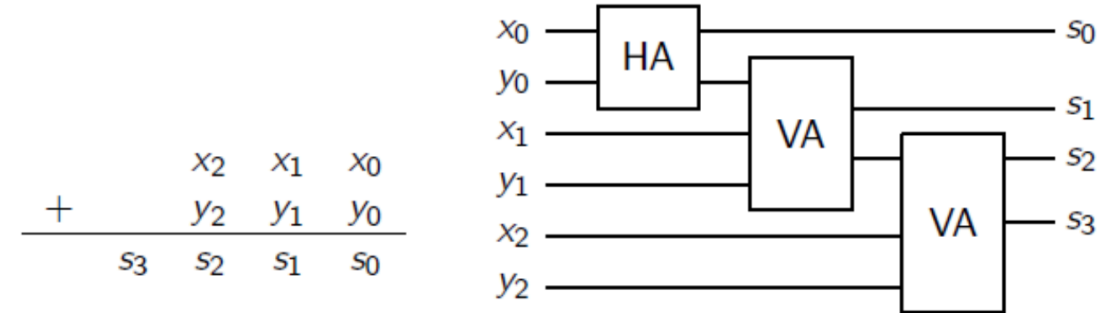
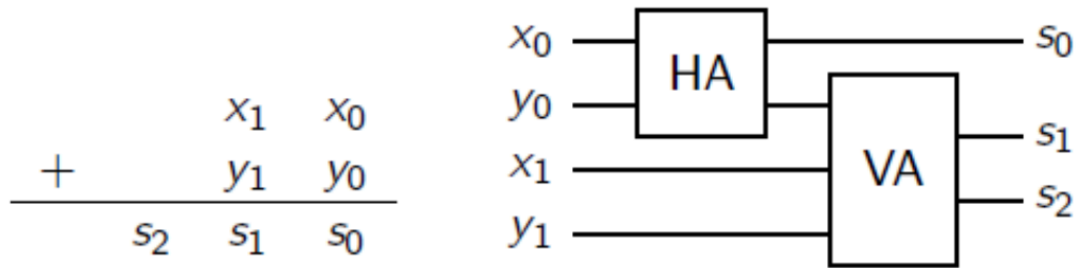


$$\begin{array}{r} + \\ \hline s_2 \quad s_1 \quad s_0 \end{array}$$

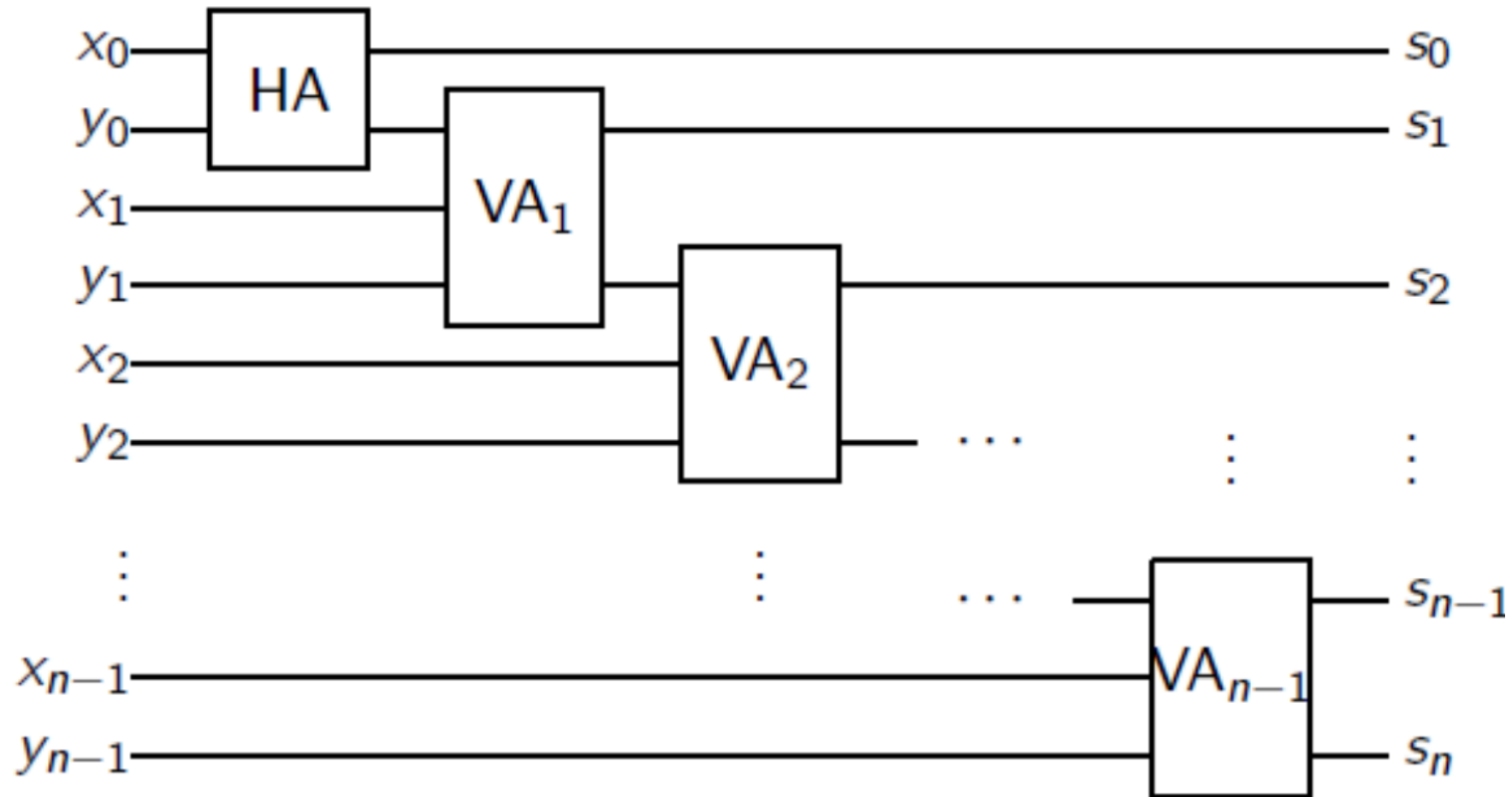


Addition natürlicher Zahlen

Vollständiger Addierer



Natürliche Zahlen addieren



Größe $2 + (n - 1) * 5 = 5n - 3$

Tiefe $1 + (n - 1) * 3 = 3n - 2$

Addition natürlicher Zahlen

Ergebnis: Ripple-Carry Addierer

Realisierung “Addition von natürlichen Zahlen”

- Sehr gut strukturiert
- Größe $5n - 3$ **sehr klein**
- Tiefe $3n - 2$ **viel zu tief**

Warum ist unser Schaltnetz so tief?

- Offensichtlich gilt: Überträge brauchen sehr lange
- Verbesserungsidee: **Überträge früh berechnen**
- Wie? Struktureinsicht

Addition natürlicher Zahlen

Struktureinsicht

- $(x_i, y_i) = (1, 1)$ generiert Übertrag
- $(x_i, y_i) = (0, 0)$ eliminiert Übertrag
- $(x_i, y_i) \in \{(0, 1), (1, 0)\}$ reicht Übertrag weiter

Dieses Verhalten ist sehr gut durch einen **Halbaddierer** realisierbar

- $u_i = 1$ generiert Übertrag
- $v_i = 1$ reicht Übertrag weiter

Zentrale Beobachtung

- Jeder HA in **Tiefe 1** parallel realisierbar berechnet u_i und v_i

x_i	y_i	u_i	v_i	Übertrag
0	0	0	0	eliminieren
0	1	0	1	weiterreichen
1	0	0	1	weiterreichen
1	1	1	0	generieren

Addition natürlicher Zahlen

Carry Look-Ahead Addierer

Notation

- $f_{HA} : B^2 \rightarrow B^2, f_{HA}(x_i, y_i) = (u_i, v_i)$
- $u_i = 1$ generiert Übertrag, $v_i = 1$ reicht Übertrag weiter

Vorausberechnung der Überträge

$$c_i = u_{i-1} \vee c_{i-1} v_{i-1}$$

Addition natürlicher Zahlen

Carry Look-Ahead Addierer

Notation

- $f_{HA} : B^2 \rightarrow B^2, f_{HA}(x_i, y_i) = (u_i, v_i)$
- $u_i = 1$ generiert Übertrag, $v_i = 1$ reicht Übertrag weiter

Vorausberechnung der Überträge

$$c_1 = u_0$$

$$c_2 = u_1 \vee u_0 v_1$$

$$c_3 = u_2 \vee u_1 v_2 \vee u_0 v_1 v_2$$

$$c_4 = u_3 \vee u_2 v_3 \vee u_1 v_2 v_3 \vee u_0 v_1 v_2 v_3$$

$$c_i = u_{i-1} \vee u_{i-2} v_{i-1} \vee u_{i-3} v_{i-2} v_{i-1} \vee \dots \vee u_0 v_1 v_2 \dots v_{i-1}$$

$$c_i = \bigvee_{j=0}^{i-1} (u_j \wedge \bigwedge_{k=j+1}^{i-1} v_k)$$

Addition natürlicher Zahlen

Carry Look-Ahead Addierer

$$c_i = \bigvee_{j=0}^{i-1} \left(u_j \wedge \bigwedge_{k=j+1}^{i-1} v_k \right)$$

Gesamttiefe des CLA: 4

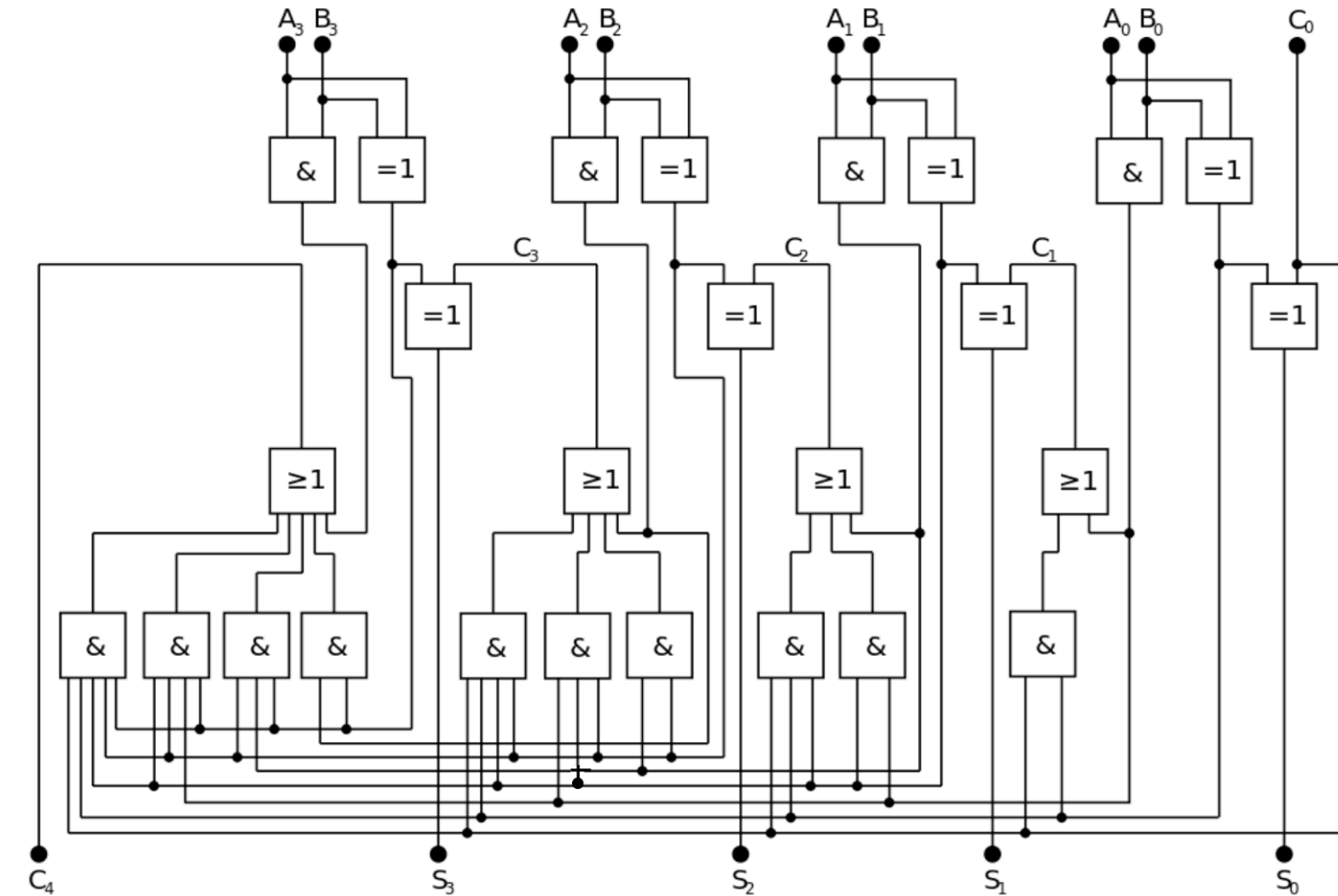
- Alle Halbaddierer (u_i, v_i) **Tiefe 1**
- Alle Und-Gatter $\left(u_j \wedge \bigwedge_{k=j+1}^{i-1} v_k \right)$ **Tiefe 1**
- Großes Oder-Gatter **Tiefe 1**

- $n \times \oplus$ -Gatter für korr. Summenbits

Tiefe 1

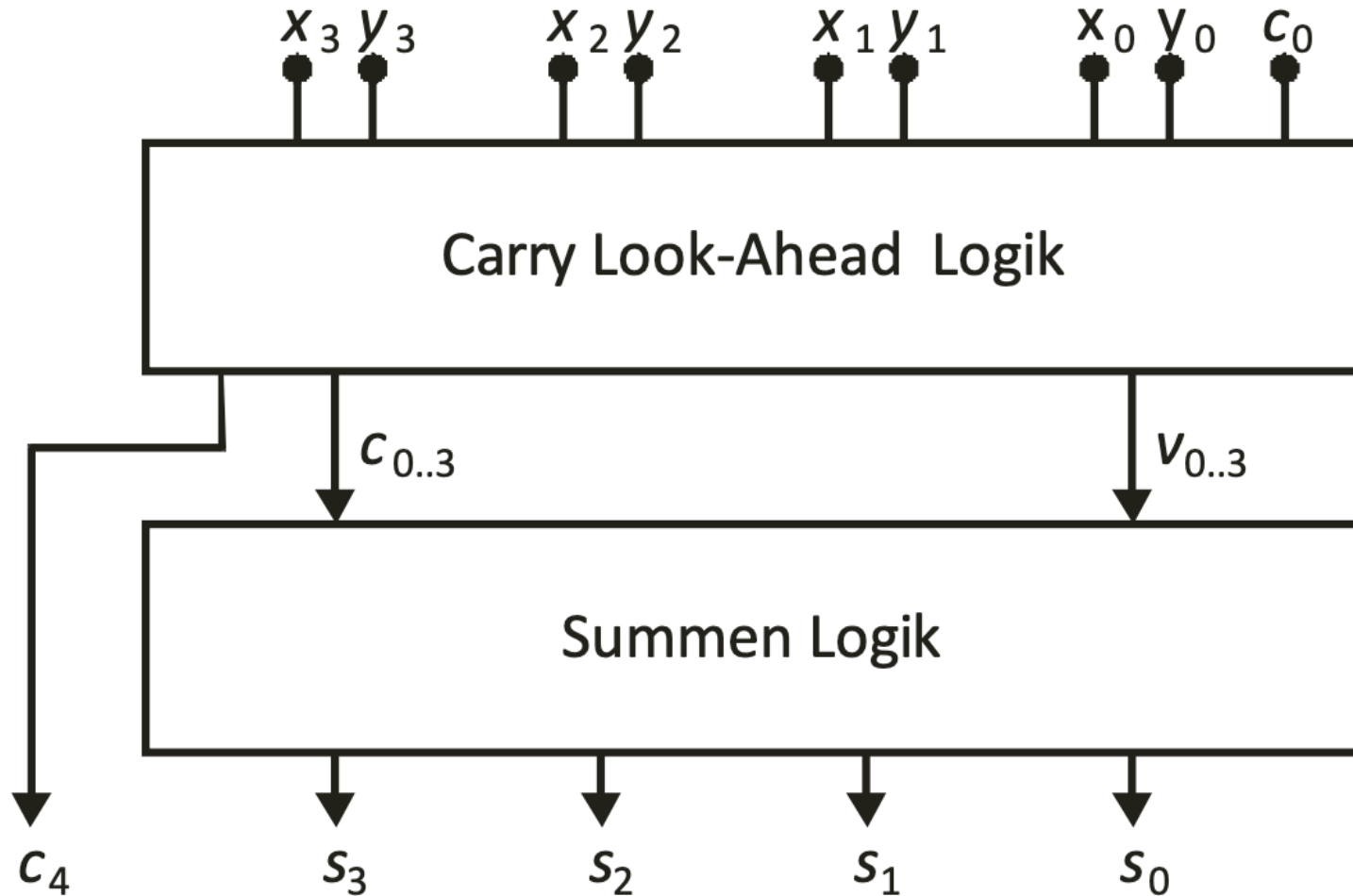
Addition natürlicher Zahlen

Carry Look-Ahead Addierer



Addition natürlicher Zahlen

Carry Look-Ahead Addierer



Addition natürlicher Zahlen

Carry Look-Ahead Addierer

Waren wir wirklich fair bei der Bestimmung der Ebenen?

- Beliebig lange Zahlen in Tiefe 4 addieren...
- Anzahl Eingänge eines Gatters heißt **Fan-In**

Nehmen wir an, dass wir nur ODER Gatter mit Fan-In 2 haben. Wie viele Gatter brauchen wir um ein ODER Logik mit Fan-In 16 zu implementieren ?

A: 1

B: 4

C: 7

D: 8

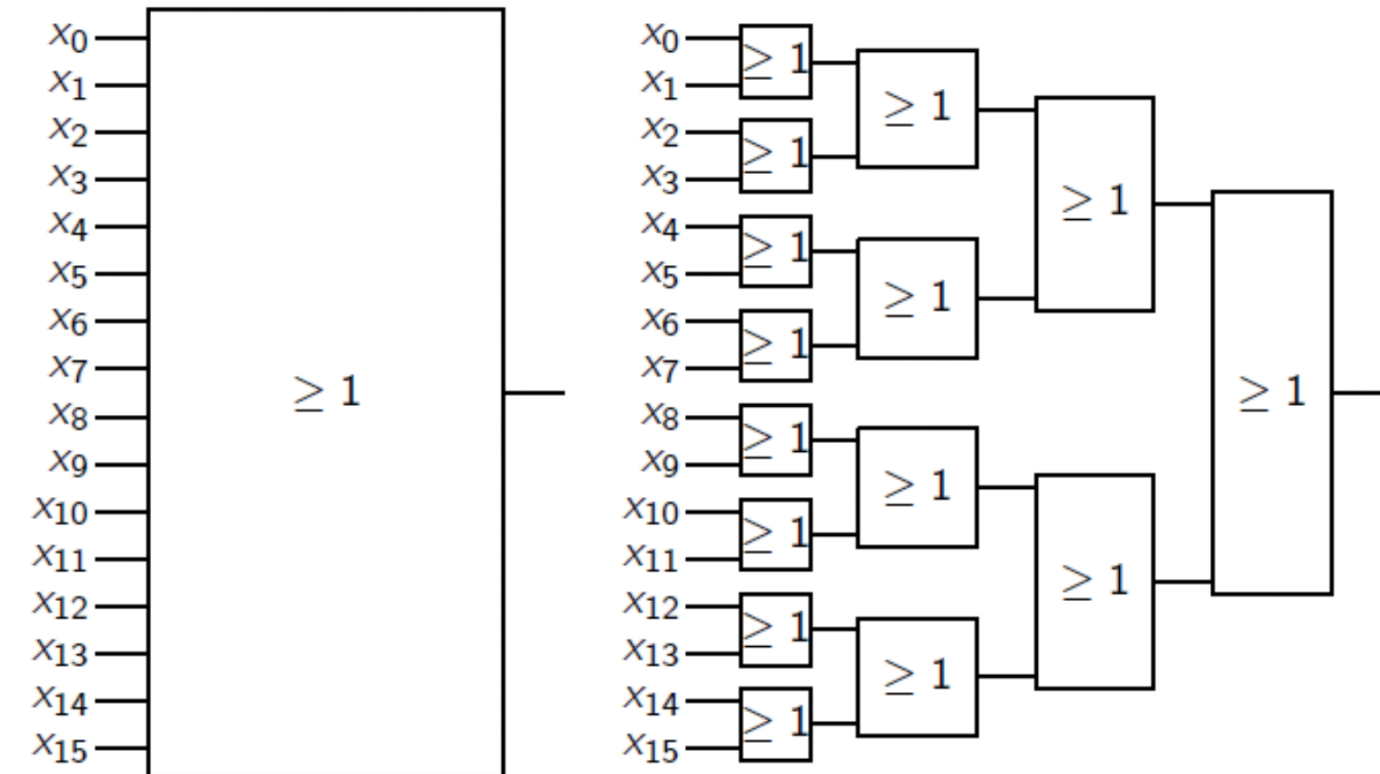
E: 15

F: 16

Addition natürlicher Zahlen

Vermeidung von großem Fan-In

Großes Fan-In **systematisch** verkleinern



Addition natürlicher Zahlen

Carry Look-Ahead Addierer

Waren wir wirklich fair bei der Bestimmung der Ebenen?

- Beliebige lange Zahlen in Tiefe 4 addieren...
- Anzahl Eingänge eines Gatters heißt **Fan-In**
- Wir vergleichen Schaltnetz mit
 - Max. Fan-In 2 (**Ripple-Carry Addierer**)
 - Mit Schaltnetz mit max. Fan-In n (**Carry Look-Ahead Addierer**)
- Große Fan-Ins sind technologisch schwierig zu realisieren

Ziel: Vergleichbarkeit herstellen

Addition natürlicher Zahlen

Vermeidung von großem Fan-In

Am Beispiel

- Aus Fan-In 16 bei Tiefe 1 (Größe 1)
- Wird Fan-In 2 bei Tiefe 4 (Größe 15)

Wie lässt sich das verallgemeinern?

- **Größe**
 - Bei jeder Stufe halb so viele Gatter wie in der Stufe davor
 - $\frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + 1 \approx n$ (*geometrische Reihe*)
- **Tiefe**
 - So oft halbieren, bis man bei einem Gatter ist

- $\approx \log_2(n)$

Addition natürlicher Zahlen

Carry Look Ahead Addierer (CLA)

- Fokus auf AND Gatter:

$$\sum_{i=1}^n \sum_{j=1}^i (j - 1)$$

- Größe $\approx \frac{n^3}{6}$
- Tiefe $\approx 2\log_2(n)$

Wolfram Alpha

Addition natürlicher Zahlen

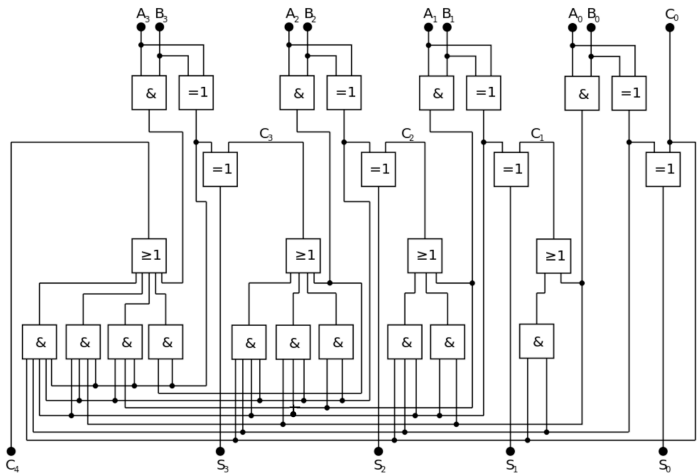
Carry look-Ahead Addierer (CLA)

Realisierung der Addition von natürlichen Zahlen

- Gut strukturiert
- Größe $\approx \frac{n^3}{6}$ $\leftarrow$ ziemlich groß
- Tiefe $\approx 2\log_2(n)$ $\leftarrow$ ziemlich flach

Vergleich zum Ripple Carry Addierer

n	Ripple-Carry Größe	Ripple-Carry Tiefe	Carry Look-Ahead Größe	Carry Look-Ahead Tiefe
8	37	22	170	6
16	77	46	1365	8
32	157	94	10922	10



n	Ripple-Carry Größe	Ripple-Carry Tiefe	Carry Look-Ahead Größe	Carry Look-Ahead Tiefe
64	317	190	87381	12

Addition natürlicher Zahlen

Welcher Addierer ist besser?

A: Ripple Carry Addierer

B: Carry Look Ahead Addierer

Addition natürlicher Zahlen



Darstellung der Tiefe / Größe

Addition natürlicher Zahlen

Einschub - Wie entsteht eine digitale Schaltung?

Video der Firma ELMOS, Dortmund

ELMOS: Wie entsteht ein Halbleiter-Chip?



<http://www.youtube.com/watch?v=kuANgMCRnqY>

Rechnerarithmetik

1. Rechnerarithmetik

1. Einleitung ✓
2. Addition natürlicher Zahlen ✓
3. **Multiplikation natürlicher Zahlen**
4. Addition ganzer Zahlen
5. Addition von Fließkommazahlen
6. Multiplikation von Fließkommazahlen

Multiplikation

direkt mit Binärzahlen...

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1\ 1\ 0\ 0\ .\ 1\ 1\ 1\ 0\ 1\ 0 \\ \hline 0 \\ 1\ 0\ 1\ 0\ 1\ 1\ 0\ 0 \\ 0 \\ 1\ 0\ 1\ 0\ 1\ 1\ 0\ 0 \\ 1\ 0\ 1\ 0\ 1\ 1\ 0\ 0 \\ 1\ 0\ 1\ 0\ 1\ 1\ 0\ 0 \\ \hline 1\ 0\ 0\ 1\ 1\ 0\ 1\ 1\ 1\ 1\ 1\ 0\ 0\ 0 \end{array}$$

Multiplizieren heißt

- Nullen passend schreiben
- Zahlen passend verschoben kopieren
- Viele Zahlen addieren

Multiplikation

Multiplikation als Schaltnetz

Multiplikation ist

- Nullen passend schreiben
- Zahlen passend verschoben kopieren
- Viele Zahlen addieren

Beobachtung

- Nullen schreiben **einfach** und **kostenlos**
- Zahlen verschieben und kopieren **einfach** und **kostenlos**

- **Viele** Zahlen addieren nicht ganz so einfach

Multiplikation: Addition vieler Zahlen

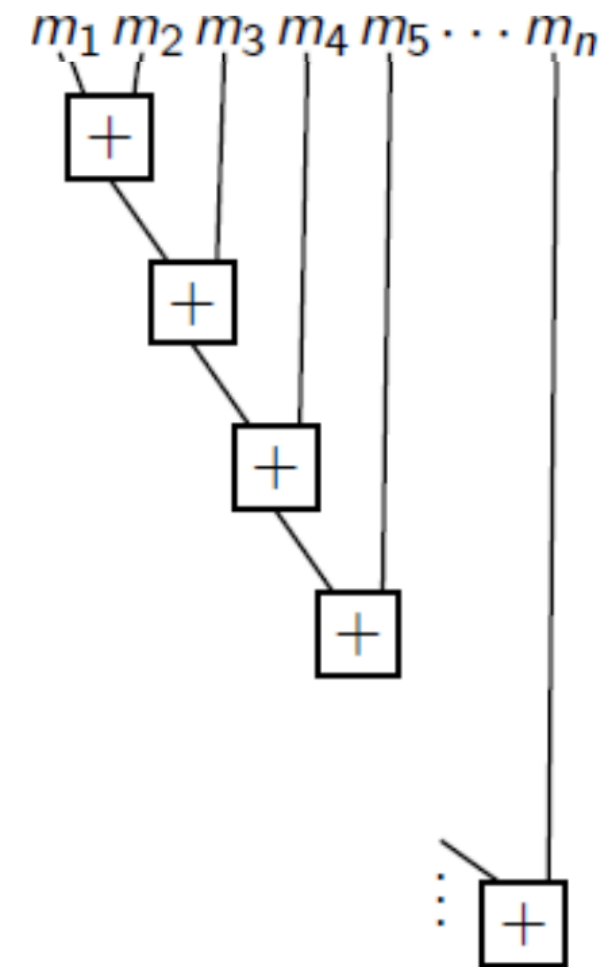
- Wir haben Addierer für die Addition zweier Zahlen
- Wie addieren wir damit n Zahlen?

Erster Ansatz Einfach nacheinander

- Schema hat die Tiefe $(n-1)$
- In jeder Ebene noch die Tiefe des Addierwerks
- Tiefe = $(n - 1) * \text{Tiefe}(\text{Addierer}) \Rightarrow$ ist zu tief!

Idee

- In Schaltnetzen **niemals** alles nacheinander tun!
- Was geht gleichzeitig?



Multiplikation: Addition vieler Zahlen

Besserer Ansatz: Paarweise addieren

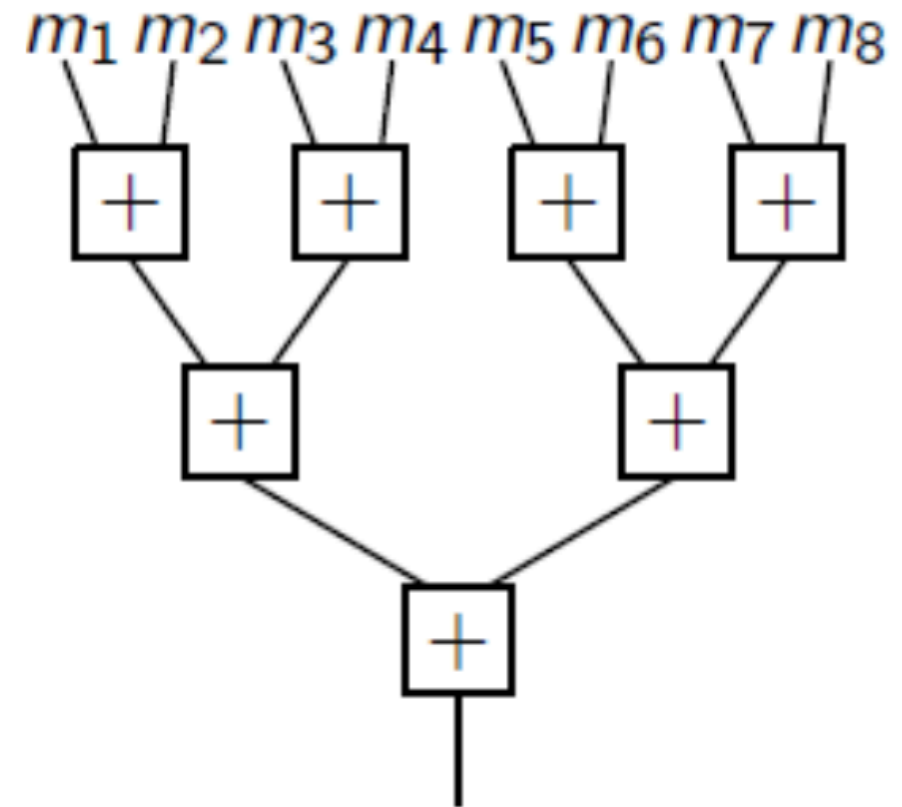
Bewertung

- Anzahl Addierer = $\frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + 1 \approx n$
- Gesamtgröße $\approx n * \text{Größe(Addierer)}$
- Tiefe $\approx \log_2(n)$ Ebenen

Gesamttiefe

$$\approx \log_2(n) * 2\log_2(n) = 2(\log_2(n))^2$$

(Anzahl der zu addierenden Bitvektoren ist gleich der Anzahl Bits)



Multiplikation: Addition vieler Zahlen

Gibt es einen noch besseren Ansatz?

Beobachtung

- Addition ersetzt **zwei** Zahlen
- Durch **eine** Zahl gleicher Summe

Zentral für uns

- Gleiche Summe → Korrektheit
- Weniger Zahlen → Fortschritt

(Verrückte?) Idee

Vielleicht ist es einfacher **drei** Zahlen zu ersetzen durch **zwei** Zahlen gleicher Summe?

Sei $x=(001)$, $y=(010)$, $z=(011)$, welche Bitvektoren a und b erfüllen die Bedingung $x+y+z = a+b$? (mehrere Auswahlmöglichkeiten)

A: $a = x + y, b = z$

B: $a = x + z, b = y$

C: $a = (011), b = (011)$

D: $a_i = x_i \oplus y_i \oplus z_i$ (for $i=0,1,2$) und $b_0 = 0$, $b_i = x_{i-1} \wedge y_{i-1} \wedge z_{i-1}$ (for $i=1,2$)

Multiplikation: Addition vieler Zahlen

Gibt es einen noch besseren Ansatz?

(Verrückte?) Idee

Vielleicht ist es einfacher **drei** Zahlen zu ersetzen durch **zwei** Zahlen gleicher Summe?

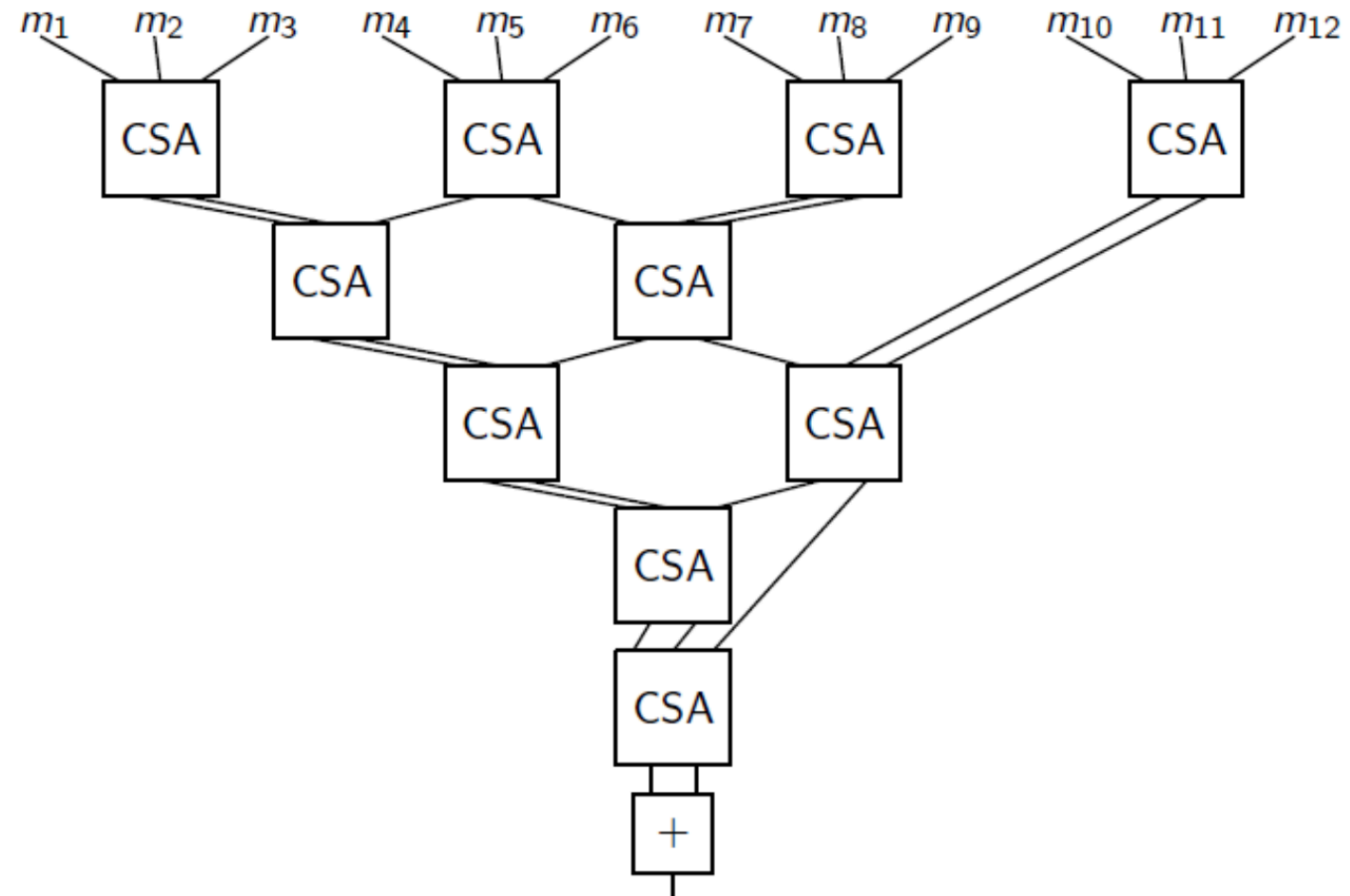
Beobachtung

- Es gilt immer noch
Gleiche Summe $\rightarrow$ Korrektheit
- **Weniger Fortschritt**
 $3 \rightarrow 2$ statt
 $2 \rightarrow 1$

Multiplikation: Addition vieler Zahlen

Wallace-Tree

- Carry Save Adder CSA
- m_i sind n-Bit Zahlen
- Sinnvoll, wenn CSA flacher ist, als CLA-Addierer



Multiplikation: Carry Save Adder

Gesucht

- Addierer, mit drei Eingängen und zwei Ausgängen, für die gilt:

$$x + y + z = a + b$$

Gefunden

- **Volladdierer**, der für $x, y, z \in \{0, 1\}$ folgendes Resultat liefert

$$x + y + z = 2 * c + s$$

- s ist das Summenbit
- $2 * c$ ist das Übertragsbit, bezogen auf seine Stelle 1 Position weiter links

Multiplikation: Carry Save Adder

- Parallelschaltung von n Volladdierern
- Liefern jeweils die **Summenbits**
- Und die **Übertragsbits**

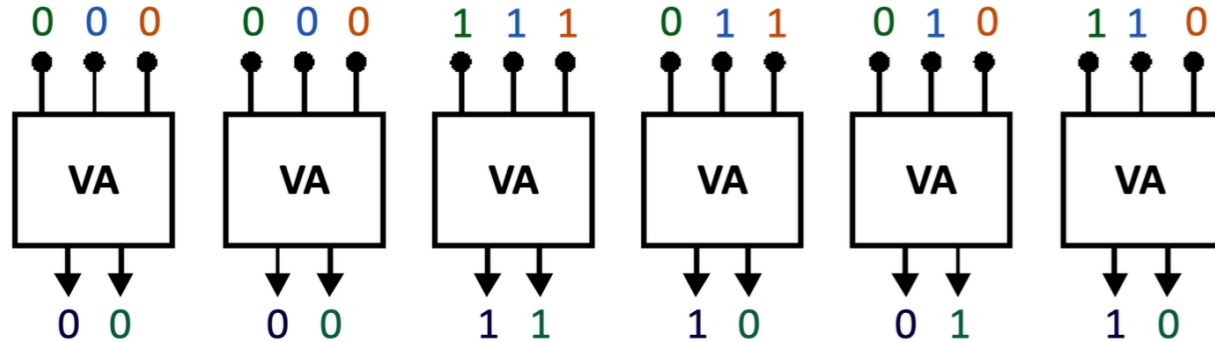
Diese Volladdierer sind nicht verkettet

- Jeder liefert ein s -Bit und ein c -Bit
- Aus diesen Bits werden
 - Das Summen-Wort gebildet, das an der höchst signifikanten Stelle (Anfang) mit 0 erweitert wird
 - Und das Carry-Wort, das an der wenigsten signifikanten Stelle (Ende) mit 0 erweitert wird
- **Summe** aus Summen-Wort und Carry-Wort **bildet das Resultat**

Multiplikation: Beispiel für Carry Save Adder

Wir addieren

- $x = 001001$
- $y = 001111$
- $z = 001100$



Summen-Wort = (0) 0 0 1 0 1 0 = 0001010
Carry-Wort = 0 0 1 1 0 1 (0) = 0011010

Multiplikation: Beispiel für Carry Save Adder

Summand x			x_5	x_4	x_3	x_2	x_1	x_0
Summand y	+		y_5	y_4	y_3	y_2	y_1	y_0
Summand z	+		z_5	z_4	z_3	z_2	z_1	z_0
Resultat			$2c_5 + s_5$	$2c_4 + s_4$	$2c_3 + s_3$	$2c_2 + s_2$	$2c_1 + s_1$	$2c_0 + s_0$
Carry-Wort		c_5	c_4	c_3	c_2	c_1	c_0	(0)
Summen-Wort	+	(0)	s_5	s_4	s_3	s_2	s_1	s_0
Summand x			0	0	1	0	0	1
Summand y	+		0	0	1	1	1	1
Summand z	+		0	0	1	1	0	0
Carry-Wort		0	0	1	1	0	1	(0)
Summen-Wort	+	(0)	0	0	1	0	1	0
Summe		0	1	0	0	1	0	0

Multiplikation

Wie viele Schritte braucht es c und s zu berechnen?

A: 1

B: 2

C: 3

D: 4

E: 5

Multiplikation: Carry Save Adder

Berechnung von drei n -Bit Zahlen

- Zum Einsatz kommen n Volladdierer
- Größe: $5n$
- Tiefe: 3

Beobachtung

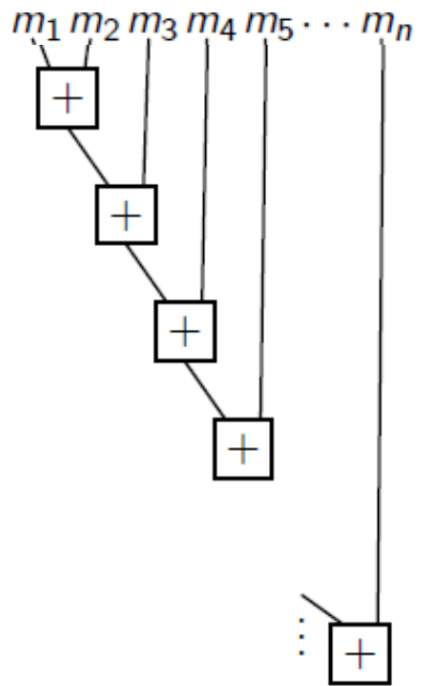
- Der Carry (große) Look-Ahead Addierer wird in einem Wallace-Tree erst als letzte Operation benötigt
- Der Wallace-Tree hat somit für die Addition von n Zahlen folgende Eigenschaften
 - Größe: $\approx \frac{n^3}{3}$ (proportional, bedingt durch den CLA)
 - Tiefe: $\approx 3\log_{3/2}(n) + 2\log_2(n) \approx 7.13\log_2(n)$

Fazit Multiplikation ist nicht **wesentlich teurer oder langsamer** als die Addition

Multiplikation

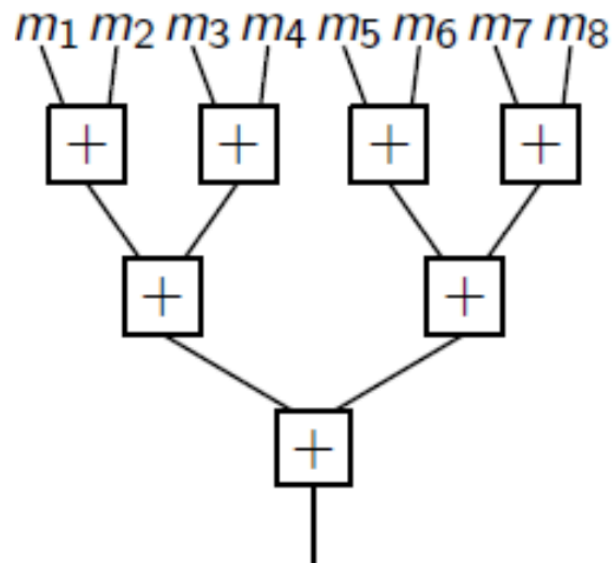
Serielle Addierer

- Tiefe = $(n - 1) * \text{Tiefe}(\text{Addierer})$
- Tiefe = $(n - 1) * 2\log_2(n)$



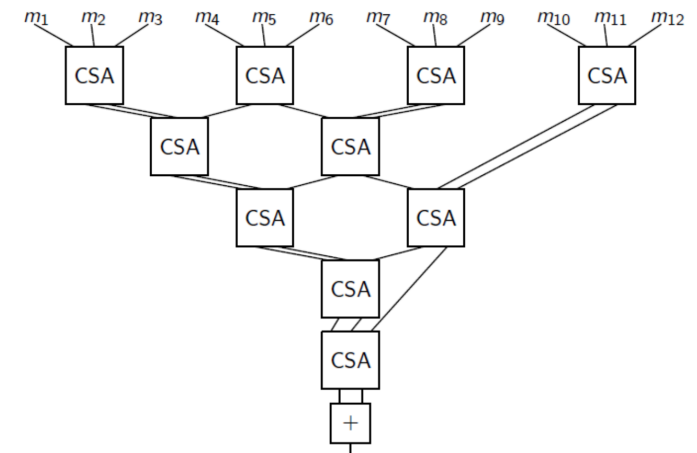
Paarweise Addierer

- Tiefe = $\log_2(n) * \text{Tiefe}(\text{Addierer})$
- Tiefe = $2(\log_2(n))^2$



Wallace-Tree mit Carry-Save Addierern

- Tiefe: $\approx 3\log_{3/2}(n) + 2\log_2(n)$
 $\approx 7.13\log_2(n)$



Multiplikation

Darstellung der Tiefe

Multiplikation

Welche Aussage ist korrekt wenn n n -bit Vektoren mit einem Wallace Tree addiert werden?

A: Der Wallace Tree benötigt nur einen Carry Look-Ahead Addierer (CLA) mit der Tiefe $2\log_2(n)$

B: Jeder Carry-Save Adder (CSA) in dem Wallace Tree ist ein n -bit Addierer der die Summe von 3 n -bit Vektoren berechnet

C: Wallace Bäume zeigen dass die Multiplikation deutlich langsamer als die Addition ist

D: Paarweise Addition ist schneller als ein Wallace Tree wenn n groß genug gewählt wird

Rechnerarithmetik

1. Rechnerarithmetik

1. Einleitung ✓
2. Addition natürlicher Zahlen ✓
3. Multiplikation natürlicher Zahlen ✓
4. **Addition ganzer Zahlen**
5. Addition von Fließkommazahlen
6. Multiplikation von Fließkommazahlen

Addition ganzer Zahlen

Addition positiver, ganzer Zahlen

- Einerkomplement
- Zweierkomplement
- Vorzeichenbetragsdarstellung
- → Addition einfach, da Zahlen binär dargestellt werden, evtl. Überträge beachten

Addition von Zahlen in Exzessdarstellung

- Für Exzessdarstellung funktioniert Addierer selbst bei positiven Zahlen **nicht**

$$x + y = (b + x) + (b + y) = (b + x + y) + b$$

→ Exzessdarstellung fürs Rechnen **weitgehend ungeeignet**, nur günstig für Vergleiche

Addition ganzer Zahlen

Warum ist die Addition ganzer Zahlen wichtig?

Beobachtung Niemand muss subtrahieren!

- Statt “ $x-y$ ” einfach “ $x + (-y)$ ” rechnen!
- **Idee** Ersetze Subtraktion durch
 1. Vorzeichenwechsel
 2. **Addition** einer eventuell negativen Zahl

Bleibt noch zu untersuchen

1. Wie schwierig ist der Vorzeichenwechsel?
2. Wie funktioniert die Addition von negativen Zahlen?

Addition ganzer Zahlen

Vorzeichenwechsel

Repräsentation	Vorgehen	Kommentar
Vorzeichen-Betrag	Vorzeichen-Bit invertieren	Sehr einfach
Einerkomplement	Alle Bits invertieren	Einfach
Zweierkomplement	Alle Bits invertieren, 1 addieren	Machbar

Grundsätzlich ist ein Vorzeichenwechsel machbar

Addition ganzer Zahlen

Addition negativer, ganzer Zahlen

Beobachtung

- Ripple-Carrier Addierer
- Carry Look-Ahead Addierer
- Sind für Betragszahlen entworfen worden

Muss die Hardware für die Addition negativer, ganzer Zahlen neu entworfen werden?

- **Exzessdarstellung** Betrachten wir gar nicht, weil wir damit nicht einmal addieren können

- **Vorzeichen-Betrag** Positive und negative fast gleich dargestellt, darum neuer Schaltnetzentwurf erforderlich

Addition ganzer Zahlen

Addition negativer Zahlen (ℓ bit) im Zweierkomplement

Notation $\bar{Y}$ ist Komplement von y

Beobachtung
$$y + \bar{Y} = 2^\ell - 1$$
$$\Leftrightarrow \bar{Y} = 2^\ell - 1 - y$$

Rechnung

$$x - y = x + (-y) = x + \bar{Y} + 1 = x + 2^\ell - 1 - y + 1 = 2^\ell + (x - y)$$

Beobachtung

- Darstellungslänge $\rightarrow 2^\ell$ “passt nicht” (Überlauf)
- Überlauf ignorieren $\rightarrow$ **Rechnung korrekt**
- Addierer rechnet **richtig** auch für negative Zahlen

$\rightarrow$ Wir benötigen keine neue Hardware, nur ein paar Regeln

Addition ganzer Zahlen

Angenommen Zahlen sind 8 bit lang und y ist eine positive Zahl. Was ist richtig wenn wir die Zweierkomplementdarstellung benutzen? (mehrere Auswahlmöglichkeiten)

A: $y + (-y) = (00000000)_2$ und das Übertragsbit an Stelle neun 0 ist.

B: $y + (-y) = (00000000)_2$ und das Übertragsbit an Stelle neun 1 ist.

C: $y +$ eine negative Zahl ist immer gültig.

D: $y +$ eine positive Zahl ist immer gültig.

Addition ganzer Zahlen

Addition negativer Zahlen (ℓ bit) im Einerkomplement

Auf dieser und nächster Folie

- Notation $\bar{Y}$ ist Komplement von y
- Wir wechseln frei zwischen Zahlen und Repräsentationen
- Feste Darstellungslänge ℓ

Beobachtung $y + \bar{Y} = 2^\ell - 1$
 $\Leftrightarrow \bar{Y} = 2^\ell - 1 - y$

Rechnung

$$x - y = x + (-y) = x + \bar{Y} = x + 2^\ell - 1 - y = 2^\ell + (x - y) - 1$$

Beobachtung

- Darstellungslänge $\rightarrow 2^\ell$ “passt nicht” (Überlauf)

- Überlauf ignorieren → Noch 1 addieren **Rechnung korrekt**

Addition ganzer Zahlen

Überträge bei Addition im Zweierkomplement

Wann ist das Ergebnis korrekt und wann nicht darstellbar?

1. Addition zweier positiver Zahlen

- Ergebnis positiv
- Kein Überlauf möglich
- **Ergebnis korrekt, wenn es positiv ist**

2. Addition einer positiven und einer negativen Zahl

- Ergebnis kleiner als größte darstellbare Zahl
- Ergebnis größer als kleinste darstellbare Zahl
- **Ergebnis immer korrekt**

3. Addition zweier negativer Zahlen

- Überlauf entsteht (ignorieren)
- Ergebnis muss negativ sein
- **Ergebnis korrekt, wenn es negativ ist**

Rechnerarithmetik

1. Rechnerarithmetik

1. Einleitung ✓
2. Addition natürlicher Zahlen ✓
3. Multiplikation natürlicher Zahlen ✓
4. Addition ganzer Zahlen ✓
5. **Addition von Fließkommazahlen**
6. Multiplikation von Fließkommazahlen

Addition von Fließkommazahlen

Fließkommazahlen

Darstellung gemäß IEEE 754-1985

$$\begin{aligned}x &= (-1)^{s_x} * m_x * 2^{e_x} & y &= (-1)^{s_y} * m_y * 2^{e_y} \\z &= x + y \\&= (-1)^{s_z} * m_z * 2^{e_z}\end{aligned}$$

- **s** Vorzeichenbit
- **m** Mantisse (Binärdarstellung, **inklusive** impliziter 1)
- **e** Exponent (Exzessdarstellung, $b = 2^{\ell_e - 1} - 1$)

Beobachtung

- Beobachtung ist einfach, wenn $e_x = e_y = e$ und $s_x = s_y = s$ gilt: $z = (-1)^s * (m_x + m_y) * 2^e$

Addition von Fließkommazahlen

Darstellung gemäß IEEE 754-1985

$$\begin{aligned}x &= (-1)^{s_x} * m_x * 2^{e_x} & y &= (-1)^{s_y} * m_y * 2^{e_y} \\z &= x + y \\&= (-1)^{s_z} * m_z * 2^{e_z}\end{aligned}$$

- **s** Vorzeichenbit
- **m** Mantisse (Binärdarstellung, **inklusive** impliziter 1)
- **e** Exponent (Exzessdarstellung, $b = 2^{\ell_e - 1} - 1$)

Angenommen es gibt 2 IEEE 754-1985 Zahlen x und y und wir berechnen $z = x + y$;
Wenn $s_x = s_y$ und $e_x = e_y$, welche Werte kann e_z annehmen? (kann mehrere
Auswahlmöglichkeiten sein)

A: e_x

B: $e_x + 1$

C: $e_x - 1$

D: beliebig

Addition von Fließkommazahlen

Darstellung gemäß IEEE 754-1985

$$x = (-1)^{s_x} * m_x * 2^{e_x} \quad y = (-1)^{s_y} * m_y * 2^{e_y} \\ z = x + y \\ = (-1)^{s_z} * m_z * 2^{e_z}$$

- **s** Vorzeichenbit
- **m** Mantisse (Binärdarstellung, inklusive impliziter 1)
- **e** Exponent (Exzessdarstellung, $b = 2^{l-1} - 1$)

Angenommen es gibt 2 IEEE 754-1985 Zahlen x und y und wir berechnen $z = x + y$;
Wenn $s_x = s_y$ und $e_x \neq e_y$ ($e_x \geq e_y$), welche Werte kann e_z annehmen? (kann mehrere Auswahlmöglichkeiten sein)

A: e_x

B: $e_x + 1$

C: $e_x - 1$

D: beliebig

Addition von Fließkommazahlen

Algorithmus zur Addition

Ziel: Berechne $z = x + y$

Ohne Beschränkung der Allgemeinheit: Sei $e_x \geq e_y$
 $\Rightarrow$ Sonst werden x und y vertauscht

Wir unterscheiden zwei Fälle:

1. $s_x = s_y$: Beide Zahlen haben das gleiche Vorzeichen
2. $s_x \neq s_y$: Die Zahlen haben unterschiedliche Vorzeichen

Generelles Problem: Bei der Addition können signifikante Bits verloren gehen!

Addition von Fließkommazahlen

Algorithmus zur Addition

Fall 1: $s_x = s_y$ (gleiches Vorzeichen)

$$z = x + y = (-1)^{s_z} [m_x 2^{e_x} + m_y 2^{e_y}] = (-1)^{s_x} * 2^{e_x} [m_x + m_y 2^{e_y - e_x}]$$

Für z gilt:

1. $s_z := s_x$

2. $e_z := e_x$

3. $m_z := m_x + m_y 2^{e_y - e_x}$

Falls $m_z \geq 2$: Ergebnis normalisieren

1. $e_z := e_z + 1$

$$2. \ m_z := \frac{m_z}{2}$$

Addition von Fließkommazahlen

Algorithmus zur Addition

Fall 2: $s_x \neq s_y$ (unterschiedliche Vorzeichen)

$$z = x + y = (-1)^{s_z} [m_x 2^{e_x} - m_y 2^{e_y}] = (-1)^{s_x} * 2^{e_x} [m_x - m_y 2^{e_y - e_x}]$$

Angenommen es gibt 2 IEEE 754-1985 Zahlen x und y und wir berechnen $z = x + y$;

Wenn $s_x \neq s_y$ und $e_x = e_y$, welche Werte kann $m_x - m_y$ (Mantisse $m_x \geq m_y$) annehmen? (kann mehrere Auswahlmöglichkeiten sein)

A: $m_x - m_y > 1$

B: $m_x - m_y < 1$

C: $m_x - m_y > 0$

D: $m_x - m_y = 0$

E: $m_x - m_y < 0$

Addition von Fließkommazahlen

Algorithmus zur Addition

Fall 2: $s_x \neq s_y$ (unterschiedliche Vorzeichen)

$$z = x + y = (-1)^{s_z} [m_x 2^{e_x} - m_y 2^{e_y}] = (-1)^{s_x} * 2^{e_x} [m_x - m_y 2^{e_y - e_x}]$$

Für z gilt:

1. $s_z := s_x$

2. $e_z := e_x$

3. $m_z := m_x - m_y 2^{e_y - e_x}$

Falls $m_z = 0$: $e_z := e_{min} - 1$

Sonst: $1 \leq m_z * 2^q < 2$ für eine ganze Zahl q

$e_z := e_z - q$ und $m_z := m_z * 2^q$

Addition von Fließkommazahlen

Algorithmus zur Addition

Fall 2: $s_x \neq s_y \Rightarrow x - y \Rightarrow$ Umwandlung von y ins Zweierkomplement

Beispiel: $m_y =$ 1.010100000000000000000000

Vorzeichenerweiterung: 01.010100000000000000000000

Einerkomplement + 10.101100000000000000000000

0.0...1:

Wie berechnen wir $-m_y 2^{e_y - e_x}$?

Mantisse verschieben: Vorne mit Einsen auffüllen

Beispiel: $-m_y * 2^{-5} = 11, 1111\ 0101\ 1000\ 0000\ 0000\ 000$

Begründung:

$$m_y * 2^{-5} = 00, 0000\ 1010\ 1000\ 0000\ 0000\ 000$$

$$-(m_y * 2^{-5}) = 11, 1111\ 0101\ 1000\ 0000\ 0000\ 000$$

Addition von Fließkommazahlen

Beispiel Addition von Fließkommazahlen

x 1 1001 0101 111 0010 0000 0000 0000 0000
y 1 1001 0100 110 0001 1000 0000 0000 0000

$e_x \geq e_y$, Vorzeichen gleich, also zunächst nur $s_z := 1$

Mantisse m_y um $[e_x - e_y = 1]$ Stelle/-n nach rechts verschieben
→ 0,111000011

Mantissen addieren

		1,	1	1	1	0	0	1		
+	0,	1	1	1	0	0	0	0	1	1
1	0,	1	1	0	0	0	1	0	1	1

Normalisieren

- Komma um eine Stelle nach links verschieben → 1,0110001011
- Exponente um 1 vergrößern → 1001 0110

Z 1 1001 0110 011 0001 0110 0000 0000 0000

Addition von Fließkommazahlen

Noch ein Beispiel zur Addition von Fließkommazahlen

x	1	1000 0101 010 0010 0000 0000 0000 0000
y	0	1000 0100 101 1010 1000 0000 0000 0000

Es gilt: $e_x > e_y$

Da $s_x \neq s_y$ muss m_y invertiert werden

Vorzeichenwechsel bei m_y in Zweierkomplementdarstellung

X	1	1000 0101 1, 010 0010 0000 0000 0000 0000
aus	0	1000 0100 01, 101 1010 0000 0000 0000 0000
wird y	1	1000 0100 10, 010 0101 1000 0000 0000 0000

Addition von Fließkommazahlen

Noch ein Beispiel zur Addition von Fließkommazahlen (2)

x 1 1000 0101 1, 010 0010 0000 0000 0000 0000
y 1 1000 0100 10, 010 0101 1000 0000 0000 0000

jetzt e_y an e_x anpassen, m_y verschieben

x 1 1000 0101 1, 010 0010 0000 0000 0000 0000
y 1 1000 0101 1, 001 0010 1100 0000 0000 0000

z 1 1000 0101 100, 011 0100 1100 0000 0000 0000

Erinnerung “überfließende” 1 einfach ignorieren

z 1 1000 0101 0, 011 0100 1100 0000 0000 0000

Normalsieren

Komma um zwei Stellen nach rechts verschieben
Exponent zum Ausgleich um zwei verkleinern

z 1 1000 0011 101 0011 0000 0000 0000 0000

Rechnerarithmetik

1. Rechnerarithmetik

1. Einleitung ✓
2. Addition natürlicher Zahlen ✓
3. Multiplikation natürlicher Zahlen ✓
4. Addition ganzer Zahlen ✓
5. Addition von Fließkommazahlen ✓
6. **Multiplikation von Fließkommazahlen**

Multiplikation von Fließkommazahlen

Darstellung gemäß IEEE-754-1985

$$x = (-1)^{s_x} * m_x * 2^{e_x}$$

$$y = (-1)^{s_y} * m_y * 2^{e_y}$$

- **s** Vorzeichenbit
- **m** Mantisse (Binärdarstellung, **inklusive** impliziter 1)
- **e** Exponent (Exzessdarstellung, $b = 2^{l-1} - 1$)

Ergebnis

$$z = (-1)^{s_z} * m_z * 2^{e_z}$$

Vereinfachung

Wir ignorieren das Runden.

Welche dieser Aussagen trifft für s_z zu?

A: $s_z = s_x \oplus s_y$

B: $s_z = s_x \vee s_y$

C: $s_z = s_x \wedge s_y$

Multiplikation von Fließkommazahlen

$$x = (-1)^{s_x} * m_x * 2^{e_x} \quad y = (-1)^{s_y} * m_y * 2^{e_y}$$

Beobachtung $z = (-1)^{s_x \oplus s_y} * (m_x * m_y) * 2^{e_x + e_y}$

Vorgehen

1. $s_z := s_x \oplus s_y$

2. $m_z := m_x * m_y$

- Multiplikation von Betragszahlen wie gesehen,
- **Implizite Einsen nicht vergessen!**

3. $e_z := e_x + e_y$

- Addition, wegen Exzessdarstellung berechnen
- $E_x := e_x + b$ (Binärrepräsentation von e_x ist E_x)
- $E_y := e_y + b$ (Binärrepräsentation von e_y ist E_y)

In welchem Bereich liegt $m_z = m_x * m_y$?

A: $m_z < 1$

B: $m_z = 0$

C: $1 \leq m_z < 2$

D: $1 \leq m_z < 4$

Multiplikation von Fließkommazahlen

Beispiel Multiplikation von Fließkommazahlen

x	1	1000 0101	101 0000 0000 0000 0000 0000
y	1	1000 0111	110 1000 0000 0000 0000 0000

Vorzeichen $s_z = 1 \oplus 1 = 0$

Exponent Bias ist $2^{l-1} - 1 = (1000\ 0000)_2 - 1$

$$E_z := E_x + E_y - b$$

$$\begin{aligned} E_y - b &= (1000\ 0111)_2 - ((1000\ 0000)_2 - 1) \\ &= (1000\ 0111)_2 - (1000\ 0000)_2 + 1 = (111)_2 + 1 = (1000)_2 \end{aligned}$$

$$E_x + E_y - b = (1000\ 0101)_2 + (1000)_2 = (1000\ 1101)_2 \rightarrow (1000\ 1101)_2 \text{ ist vorläufiger Exponent}$$

Multiplikation von Fließkommazahlen

Beispiel Multiplikation von Fließkommazahlen

Mantisse

$$\begin{array}{r} \textcolor{blue}{1}, \textcolor{green}{1} \textcolor{green}{0} \textcolor{green}{1} \cdot \textcolor{blue}{1}, \textcolor{green}{1} \textcolor{green}{1} \textcolor{green}{0} \textcolor{green}{1} \\ \hline 1 \quad 1 \quad 0 \quad 1 \\ 1 \quad 1 \quad 0 \quad 1 \\ 1 \quad 1 \quad 0 \quad 1 \\ 0 \quad 0 \quad 0 \quad 0 \\ + 1 \quad 1 \quad 0 \quad 1 \\ \hline 1 \quad 0, \textcolor{blue}{1} \quad 1 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \end{array}$$

Normalisieren

- Komma um 1 Stelle nach links
- Exponent zum Ausgleich + 1
- Implizite Eins streichen

z 0 1000 1110 011 1100 1000 0000 0000 0000

Rechnerarithmetik

1. Rechnerarithmetik

1. Einleitung ✓
2. Addition natürlicher Zahlen ✓
3. Multiplikation natürlicher Zahlen ✓
4. Addition ganzer Zahlen ✓
5. Addition von Fließkommazahlen ✓
6. Multiplikation von Fließkommazahlen ✓

Rechnerarithmetik: Real-world numerical catastrophes

Longer video of 'Ariane 5' Rocket first l...



- Ariane 5 rocket. Ariane 5 rocket exploded 40 seconds after being launched by European Space Agency on June 4th, 1996. (http://www.youtube.com/watch?v=gp_D8r-2hwk) Maiden voyage after a decade and 7 billion dollars of research and development. Sensor reported acceleration that was so large that it caused an overflow in the part of the program responsible for recalibrating inertial guidance. 64-bit floating point number was converted to a 16-bit signed integer, and the conversion failed. This resulted in a drastic attempt to correct the nonexistent problem, leading to the end of Ariane 5.
- Patriot missile accident. On February 25, 1991 an American Patriot missile failed to track and destroy an Iraqi Scud missile. Instead it hit an Army barracks, killing 26 people. The cause was an inaccurate calculate caused by measuring time in tenth of a second. Couldn't represent $1/10$ exactly since used 24 bit floating point

- Intel FDIV Bug Error in Pentium hardwire floating point divide circuit. Discovered by Intel in July 1994, rediscovered and publicized by math professor in September 1994. Intel recall in December 1994 cost \$300 million. Another floating point bug discovered in 1997.

Rechnerarithmetik

Sehr wichtiger Artikel über Fließkommazahlen

David Goldberg (1991): What every computer scientist should know about floating-point arithmetic. ACM Computing Surveys 23(1): 5-48.

